

ЛАБОРАТОРНАЯ РАБОТА №1

«GUI, модуль Tkinter»

1. Цель работы

Ознакомление с принципами организации событийно-управляемого программирования, а также с основами организации графического интерфейса пользователя (GUI) и виджетами модуля Tkinter и со способами построения графиков функций методами этого модуля.

2. Постановка задачи и вариант задания

Вывести на экран в графическом режиме графики двух функций на интервале от $X_{нач}$ до $X_{кон}$ с шагом dx . Первая функция задана с помощью ряда Тейлора, ее вычисление должно выполняться с точностью ε . Значение параметра b , определяющего смещение функции по оси ординат, вводится с клавиатуры. Графики должны быть плавными и различаться цветами.

Вариант 4:

$$y(x) = \frac{\pi}{2} + \sum_{n=0}^{\infty} \frac{(-1)^{n+1}}{(2n+1) \cdot x^{2n+1}}, \quad x > 1$$

$$z(x) = \arctg(x) + b$$

ХОД РАБОТЫ

3. Листинг кода на языке Python

```

from tkinter import *
from tkinter.messagebox import *
import numpy as np

# ----- Функция  $y(x) = \dots$  -----
def ArctgXDevx():
    eps = 0.0001
    Lfun = []
    x = Xmin
    while x <= Xmax:
        an = -1.0 / x
        Sum = an
        n = 0
        while abs(an) > eps:
            an *= -(2 * n + 1) / ((2 * n + 3) * x * x)
            Sum += an
            n += 1
        Lfun.append((x, np.pi / 2 + Sum))
        x += 1 / Kx
    return Lfun

# ----- Вторая функция:  $z(x) = \arctg(x) + b$  -----
def ArctgXOnX():
    Lfun = []
    x = Xmin
    while x <= Xmax:
        fun = np.atan(x) + dY
        Lfun.append((x, fun))
        x += 1 / Kx
    return Lfun

def GetData():
    '''Получить данные'''
    global Xmax, Xmin, Ymax, Ymin
    global dX, dY, Kx, Ky
    try:
        tmpXmax = float(ent3.get())
        tmpXmin = float(ent2.get())
        tmpYmax = float(ent5.get())
        tmpYmin = float(ent4.get())
        tmpDY = float(ent7.get())
        tmpDX = float(ent6.get())
    except ValueError:
        showwarning(title="Ошибка ввода",
                    message="Все поля должны содержать числа")
    return

if (tmpXmin <= 1.0) or (tmpXmin >= tmpXmax) or \

```

```

(tmpYmin >= tmpYmax) or (tmpDX <= 0):
    showwarning(title="Ошибка задания границ",
                message="Должны выполняться неравенства:\n"
                        "Xmin > 1 (область сходимости ряда);\n"
                        "Xmax > Xmin;\n"
                        "Ymax > Ymin;\n"
                        "Шаг меток > 0")

    return

Xmax, Xmin = tmpXmax, tmpXmin
Ymax, Ymin = tmpYmax, tmpYmin
dX, dY = tmpDX, tmpDY
Kx = MaxX / abs(Xmax - Xmin)
Ky = MaxY / abs(Ymax - Ymin)

def SetMark(a, b, LrBt=1):
    '''Нанесение меток'''
    ax_XY = []
    if LrBt:
        # слева и справа
        ax_XY.append((a, b))
        ax_XY.append((a + 10, b))
    else:
        # сверху и снизу
        ax_XY.append((a, b))
        ax_XY.append((a, b - 10))
    cv.create_line(ax_XY, fill='black', width=2)

def plotXY():
    '''Рисуем координатные линейки'''
    # Прямоугольник
    ax_XY = [(5, 5), (MaxX - 5, MaxY - 5)]
    cv.create_rectangle(ax_XY, fill="white", outline="green",
                        width=2)

    # Разметка левой и правой сторон
    y = Ymin
    y_pix = MaxY
    flg = False
    while y < Ymax:
        textY = str(round(y, 2))
        SetMark(0, y_pix, 1)
        if flg:
            cv.create_text(15, y_pix, text=textY, anchor=W)
        SetMark(MaxX - 10, y_pix, 1)
        if flg:
            cv.create_text(MaxX - 15, y_pix, text=textY, anchor=E)
        y += dX
        y_pix -= dX * Ky
        flg = not flg

    # Разметка сверху и снизу
    x = Xmin
    x_pix = 0
    flg = False

```

```

while x < Xmax:
    textX = str(round(x, 2))
    SetMark(x_pix, 0, 0)
    if flg:
        cv.create_text(x_pix, 15, text=textX, anchor=N)
    SetMark(x_pix, MaxY, 0)
    if flg:
        cv.create_text(x_pix, MaxY - 15, text=textX, anchor=S)
    x += dX
    x_pix += dX * Kx
    flg = not flg

def Draw(event):
    '''Подготовка полотна и вызов функций для рисования'''
    cv.delete("all")
    GetData()
    plotXY()
    Fdraw(ArctgXDevx, 'blue')    # ряд Тейлора
    Fdraw(ArctgXOnX, 'red')      # arctg(x) + b
    print('Рисуем')

def Fdraw(func, color):
    '''Получение значений функции, преобразование в пиксели,
    рисование'''
    Lxy = func()
    Lpix = []
    for xy in Lxy:
        x = Kx * (xy[0] - Xmin)
        if xy[1] is not None:
            y = MaxY - Ky * (xy[1] - Ymin)
        else:
            y = 0
        Lpix.append((x, y))
    cv.create_line(Lpix, fill=color)

def Final(event):
    '''Завершение работы'''
    window_deleted()

def window_deleted():
    '''Завершение работы по [X]'''
    if askyesno("Выход", "Завершить работу?"):
        root.destroy()

def showXY(event):
    global ID1, ID2
    x = event.x
    y = event.y
    ent0.delete(0, END)
    ent1.delete(0, END)
    ent0.insert(0, str(round(Xmin + x / Kx, 2)))
    ent1.insert(0, str(round(Ymin + (MaxY - y) / Ky, 2)))
    cv.delete(ID1)

```

```

cv.delete(ID2)
ID1 = cv.create_line(0, y, MaxX, y, dash=(3, 5))
ID2 = cv.create_line(x, 0, x, MaxY, dash=(3, 5))

# ===== Основная программа =====
root = Tk()
root.title("Графика - Вариант 4")
root.protocol('WM_DELETE_WINDOW', window_deleted)
root.resizable(False, False)

Kp = 0.7
MaxX = int(root.winfo_screenwidth() * Kp)
MaxY = int(root.winfo_screenheight() * Kp)

cv = Canvas(root, width=MaxX, height=MaxY, bg="white")
cv.grid(row=0, columnspan=9)
cv.bind('<Button-1>', showXY)

# Начальные параметры (область сходимости ряда:  $x > 1$ )
Xmin, Xmax = 1.1, 10.0
Ymin, Ymax = 0.0, 2.0
dY = 0.0      # смещение b второй функции
dX = 1.0      # шаг меток на осях

ID1 = 0
ID2 = 0
Kx = MaxX / abs(Xmax - Xmin)
Ky = MaxY / abs(Ymax - Ymin)

# ----- Метки и поля ввода/вывода -----
lba0 = Label(root, text="X:", width=10, fg="blue", font="Ubuntu, 12")
lba0.grid(row=1, column=0, sticky='e')
ent0 = Entry(root, width=5, font="Ubuntu, 12")
ent0.grid(row=1, column=1)
ent0.insert(0, "0")

lba1 = Label(root, text="Y:", width=10, fg="blue", font="Ubuntu, 12")
lba1.grid(row=2, column=0, sticky='e')
ent1 = Entry(root, width=5, font="Ubuntu, 12")
ent1.grid(row=2, column=1, sticky='w')
ent1.insert(0, "0")

lba2 = Label(root, text="Xmin:", width=10, fg="blue", font="Ubuntu, 12")
lba2.grid(row=1, column=2, sticky='e')
ent2 = Entry(root, width=5, font="Ubuntu, 12")
ent2.grid(row=1, column=3)
ent2.insert(0, str(Xmin))

lba3 = Label(root, text="Xmax:", width=10, fg="blue", font="Ubuntu, 12")
lba3.grid(row=1, column=4, sticky='e')
ent3 = Entry(root, width=5, font="Ubuntu, 12")

```

```

ent3.grid(row=1, column=5)
ent3.insert(0, str(Xmax))

lba4 = Label(root, text="Ymin:", width=10, fg="blue", font="Ubuntu,
12")
lba4.grid(row=2, column=2, sticky='e')
ent4 = Entry(root, width=5, font="Ubuntu, 12")
ent4.grid(row=2, column=3)
ent4.insert(0, str(Ymin))

lba5 = Label(root, text="Ymax:", width=10, fg="blue", font="Ubuntu,
12")
lba5.grid(row=2, column=4, sticky='e')
ent5 = Entry(root, width=5, font="Ubuntu, 12")
ent5.grid(row=2, column=5)
ent5.insert(0, str(Ymax))

lba6 = Label(root, text="Шаг меток:", width=10, fg="blue",
font="Ubuntu, 12")
lba6.grid(row=1, column=6, sticky='e')
ent6 = Entry(root, width=5, font="Ubuntu, 12")
ent6.grid(row=1, column=7)
ent6.insert(0, str(dX))

lba7 = Label(root, text="Смещение b:", width=10, fg="blue",
font="Ubuntu, 12")
lba7.grid(row=2, column=6, sticky='e')
ent7 = Entry(root, width=5, font="Ubuntu, 12")
ent7.grid(row=2, column=7)
ent7.insert(0, str(dY))

btn1 = Button(root, width=20, bg="#ccc", text="Рисовать")
btn1.grid(row=1, column=8)
btn1.bind("<Button-1>", Draw)

btn2 = Button(root, width=20, bg="#ccc", text="Выход")
btn2.grid(row=2, column=8)
btn2.bind("<Button-1>", Final)

root.mainloop()

```

4. Тестирование и отладка программы

На рисунках 1.1, 1.2 и 1.3 отображены результаты работы кода согласно всем условиям по заданию.

На рисунке 1.1 На полотне отображается одна красная кривая, которая по форме соответствует графику арктангенса. Синяя кривая (ряд Тейлора) не видна, потому что при $b = 0$ обе функции математически совпадают, и красная линия просто перекрывает синюю.

На рисунке 1.2 проводится аналогичный тест программы, но из-за изменения параметра смещения $b = 0.2$ график функции $z(x) = \arctg(x) + b$ (линия красного цвета) идёт параллельно функции, описывающей ряд Тейлора для $\arctg(x)$ (линия синего цвета). Это доказывает, что ряд Тейлора сходится к функции $\arctg(x)$ с заданной точностью, а параметр смещения b работает правильно.

На рисунке 1.3 Появилось диалоговое окно «Ошибка задания границ» с текстом, сообщающим пользователю допустимые ограничения для программы.

Таким образом можно сказать, что программа корректно выполняет поставленную задачу. Она строит графики двух функций, позволяет управлять масштабом и смещением через интерфейс, обрабатывает клики мыши для вывода координат и защищена от ввода недопустимых данных.

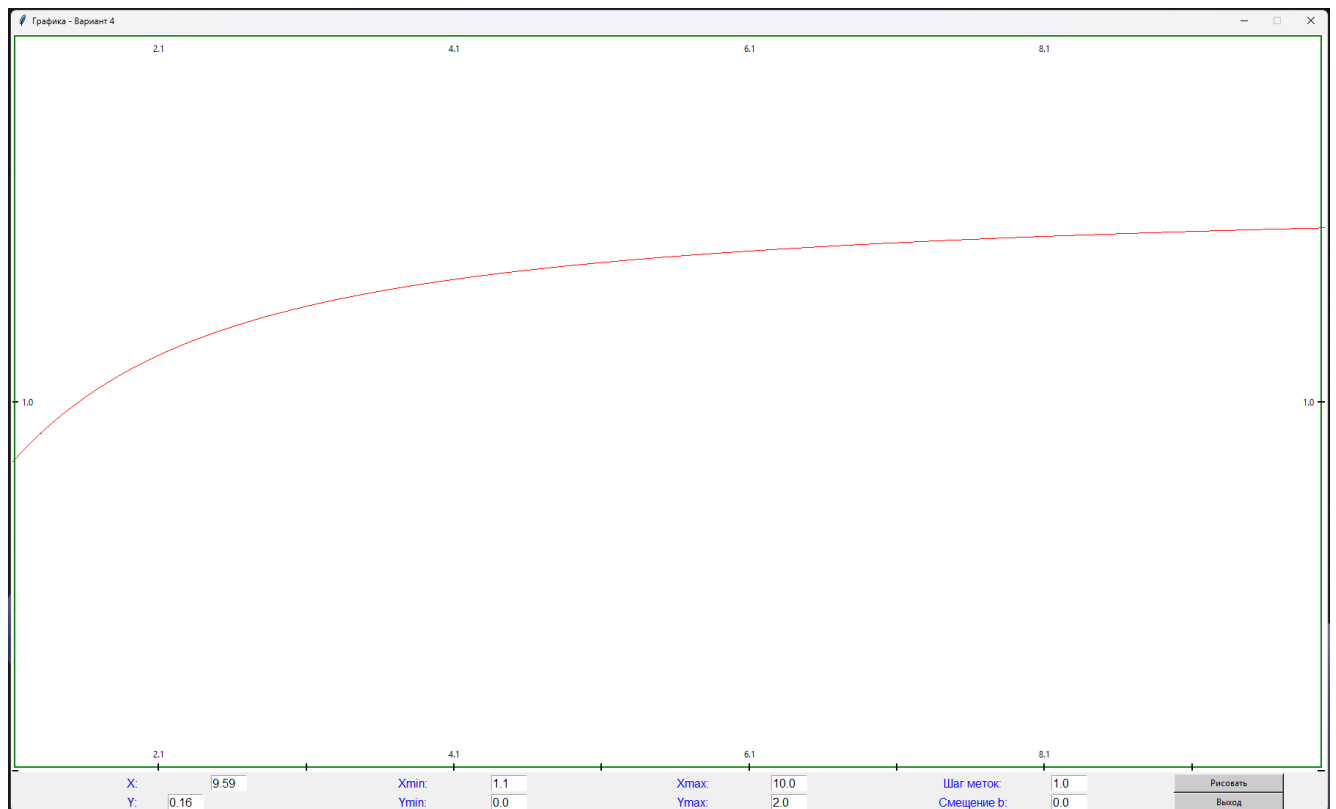


Рисунок 1.1 – Результат работы программы с параметрами по умолчанию

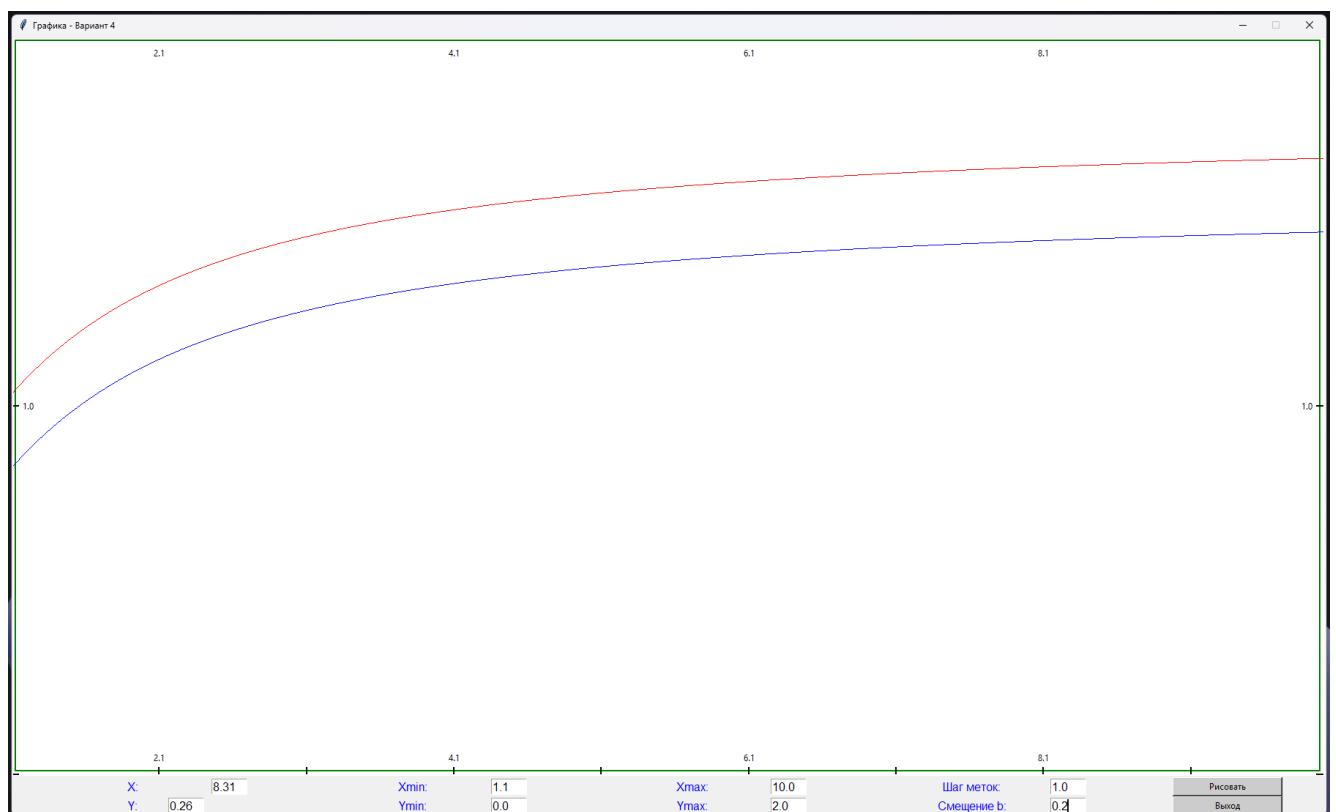


Рисунок 1.2 – Результат работы программы с изменённым параметром смещения
($b = 0.2$)

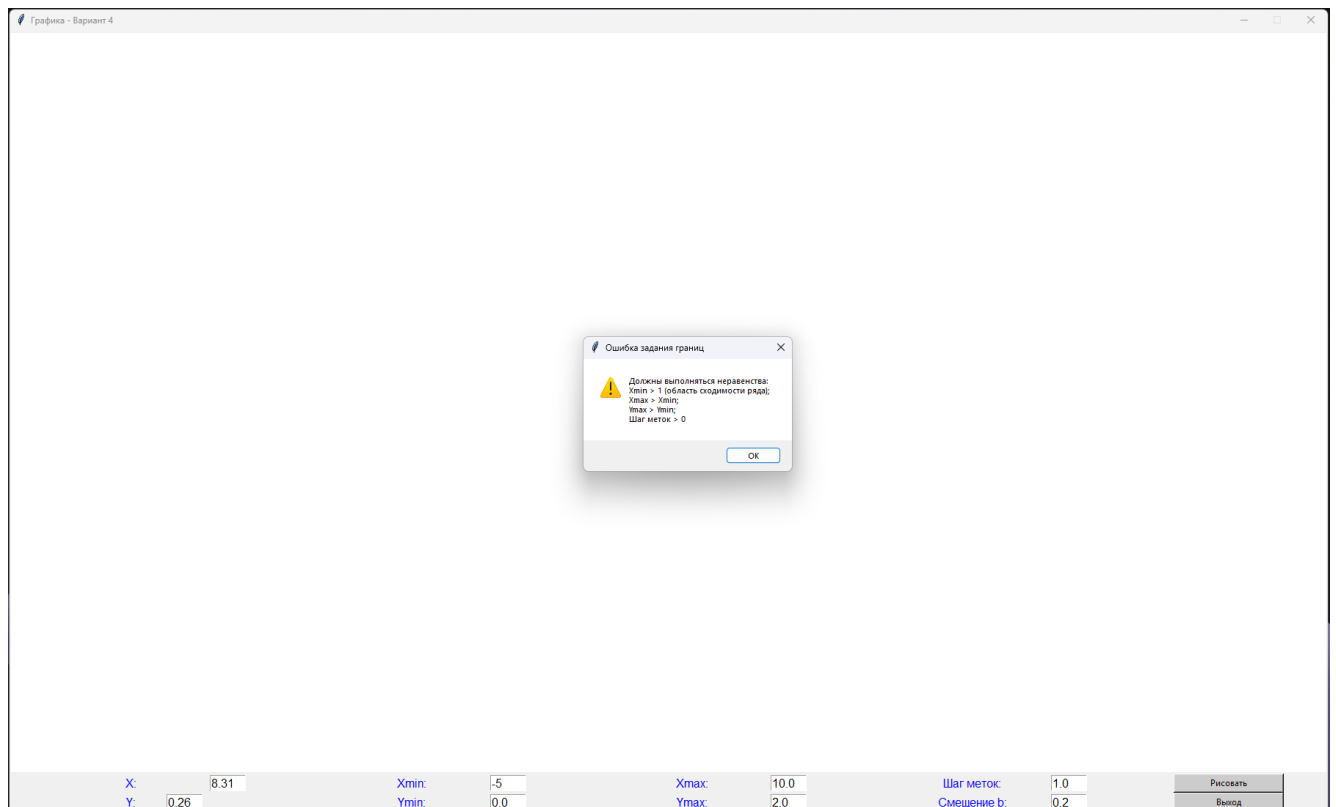


Рисунок 1.3 – Результат работы программы с подачей ошибочных параметров

ВЫВОД

В ходе выполнения лабораторной работы были изучены принципы событийно-управляемого программирования и основы построения графического интерфейса пользователя средствами модуля Tkinter, а также освоены приёмы построения графиков функций на виджете Canvas. В соответствии с вариантом №4 была разработана программа, которая строит на интервале от $X_{\min}$ до $X_{\max}$ графики двух функций: $y(x) = \frac{\pi}{2} + \sum_{n=0}^{\infty} \frac{(-1)^{n+1}}{(2n+1) \cdot x^{2n+1}}$, вычисляемой с помощью ряда Тейлора с заданной точностью $\varepsilon = 0,0001$, и $z(x) = \arctg(x) + b$, где параметр смещения b вводится пользователем с клавиатуры через виджет Entry. В программе реализованы преобразование пользовательских координат в пиксельные и обратно через масштабные коэффициенты K_x и K_y , разметка координатных линеек с подписями, вывод текущих координат курсора по клику мыши, а также проверка корректности вводимых данных, запрещающая задавать $X_{\min} \leq 1$ из-за области сходимости ряда. Тестирование подтвердило работоспособность программы: при $b = 0$ графики совпадают, при $b = 0,2$ красная кривая идёт параллельно синей, а при вводе ошибочных границ выводится предупреждающее диалоговое окно, не прерывающее работу приложения.